3008ICT Deep Learning

Week 10. Pre-training, BERT

Adapted from slides by Greg Durrett

LM Evaluation

- Accuracy doesn't make sense for language models — predicting the next word is generally impossible so accuracy values would be very low
- Evaluate LMs on the likelihood of held-out data (averaged to normalise for length)

$$\frac{1}{n}\sum_{i=1}^{n}\log P(w_i|w_1,\ldots,w_{i-1})$$

- Perplexity: exp(average negative log likelihood). Lower perplexity is better
  - Suppose $P(w_1 \mid w_0) = \frac{1}{4}$, $P(w_2 \mid w_1, w_0) = \frac{1}{3}$, $P(w_3 \mid w_2, w_1, w_0) = \frac{1}{4}$ for 3 predictions
  - Perplexity = $\exp(-\frac{1}{3}(\log\frac{1}{4} + \log\frac{1}{3} + \log\frac{1}{4})) \approx 7.268$

Scaling Law

- Scaling Law in AI: Larger models trained on more data with more compute yield better results
- The "Bitter Lesson": Simple algorithms that leverage scaling often outperform more complex, hand-engineered approaches in the long run.
-

Scaling Laws

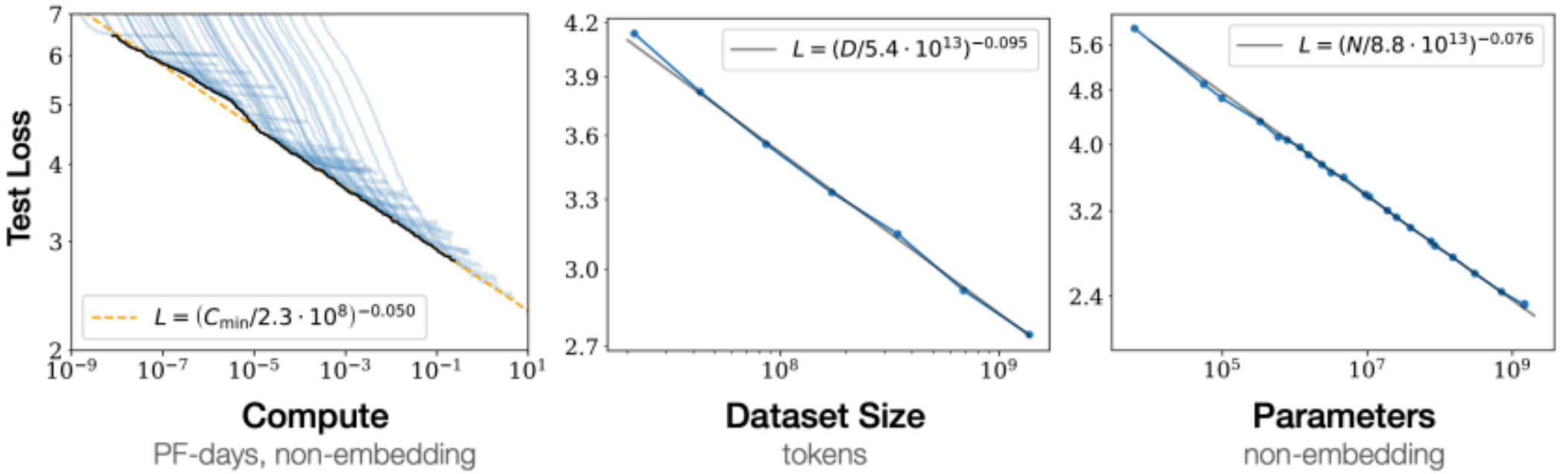


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

- Transformers scale really well!

Kaplan et al. (2020)

Takeways

- Transformers are going to be the foundation for much of modern NLP and are a ubiquitous architecture nowadays
- Many details to get right, many ways to tweak and extend them, but core idea is the multi-head self attention and their ability to contextualise items in sequences

Pretraining, ELMO

What is pre-training?

- “Pre-train” a model on a large dataset for task X, then “fine-tune” it on a dataset for task Y
- Key idea: X is somewhat related to Y, so a model that can do X will have some good neural representations for Y as well
- ImageNet pre-training is huge in computer vision: learn generic visual features for recognising objects
- GloVe can be seen as pre-training: learn vectors with the skip-gram objective on large data (task X), then fine-tune them as part of a neural network for sentiment/any other task (task Y)

GloVe is insufficient

- GloVe uses a lot of data but in a weak way
- Having a single embedding for each word is wrong

  they swing the bats *they see the bats*

  - Identifying discrete word senses is hard, doesn't scale. Hard to identify how many senses each word has
- How can we make our word embeddings more context-dependent?

ELMO

- Use the embeddings as a drop-in replacement for GloVe
  - Huge gains across many high-profile tasks: NER, question answering, semantic role labeling (similar to parsing), etc.
- But what if the pre-training **isn’t only the embeddings**?

- A deeper question: What if we pre-train the entire network?"

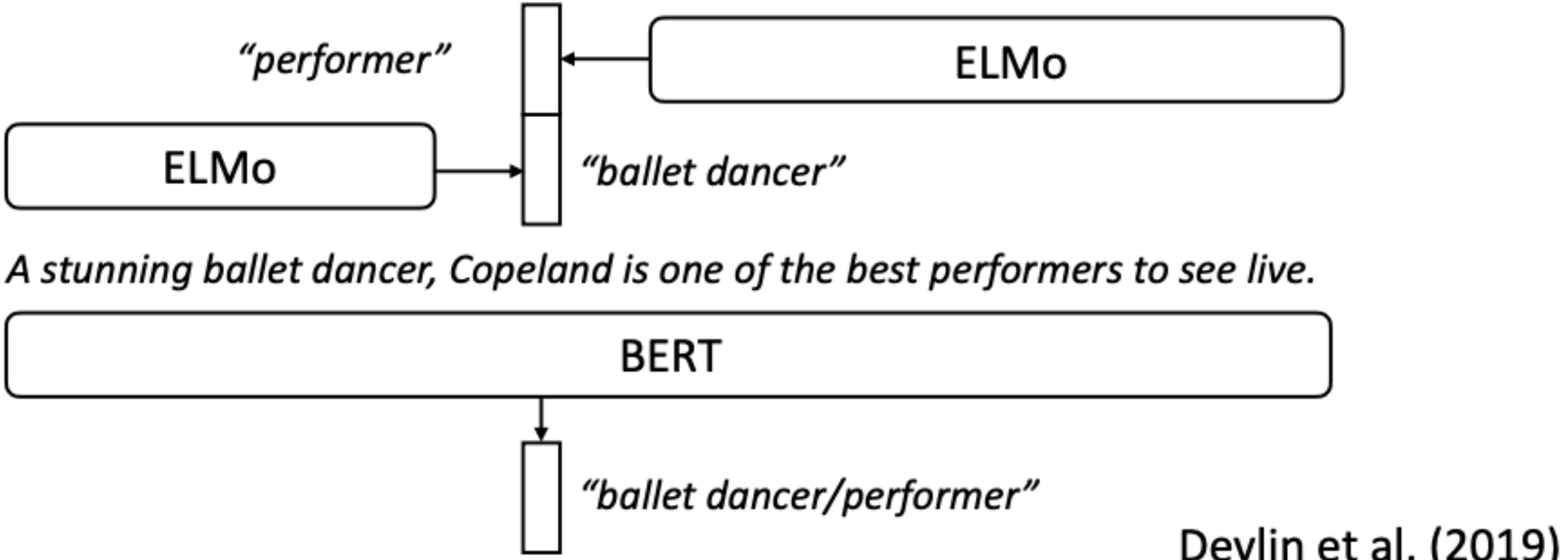
BERT

BERT

- AI2 made ELMo in spring 2018, GPT (transformer-based ELMo) was released in summer 2018, BERT came out October 2018
- Four major changes compared to ELMo:
  - Transformers instead of LSTMs
  - Bidirectional model with “Masked LM” objective instead of standard LM
  - Fine-tune instead of freeze at test time
  - Operates over word pieces (byte pair encoding)

BERT

- ELMo is a unidirectional model (as is GPT): we can concatenate two unidirectional models, but is this the right thing to do?
- ELMo reprs look at each direction in isolation; BERT looks at them jointly

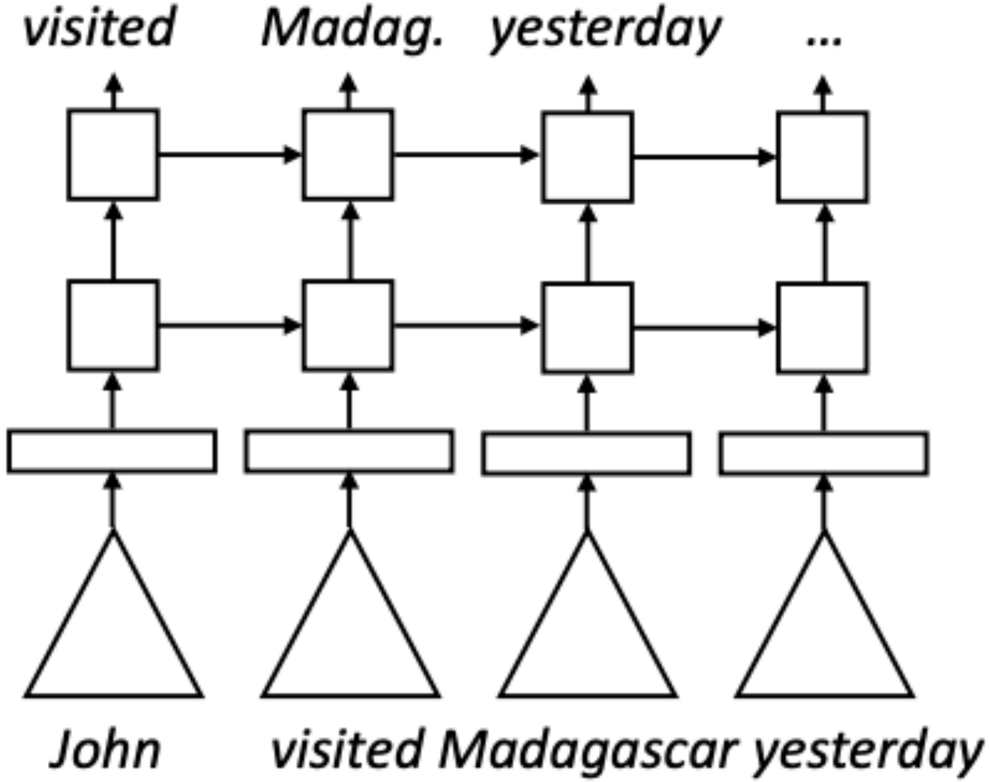


Devlin et al. (2019)

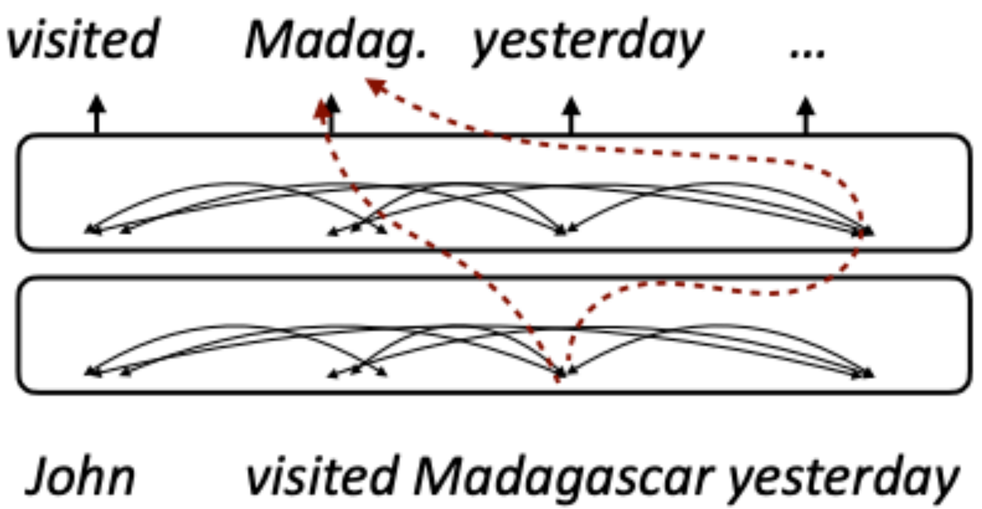
BERT

- How to learn a “deeply bidirectional” model? What happens if we just replace an LSTM with a transformer?

ELMo (Language Modeling)



BERT

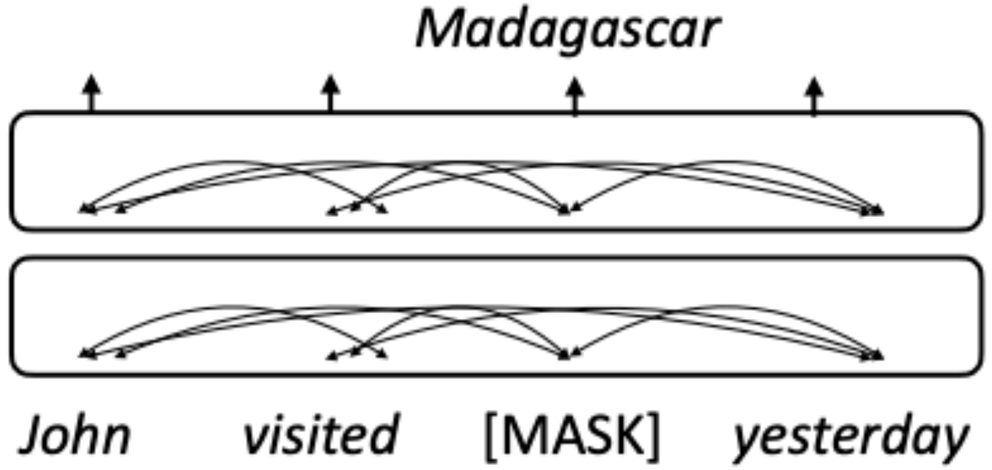


- You could do this with a “one-sided” transformer, but this “two-sided” model can cheat

Masked Language Mdeling

- How to prevent cheating? Next word prediction fundamentally doesn't work for bidirectional models, instead do *masked language modeling*

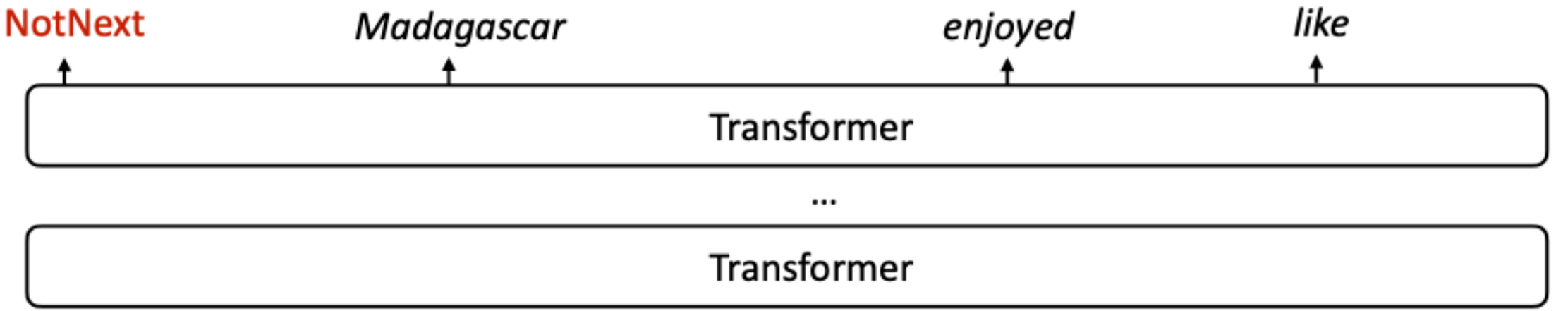
- BERT formula: take a chunk of text, mask out 15% of the tokens, and try to predict them



Devlin et al. (2019)

Next “Sentence” Prediction

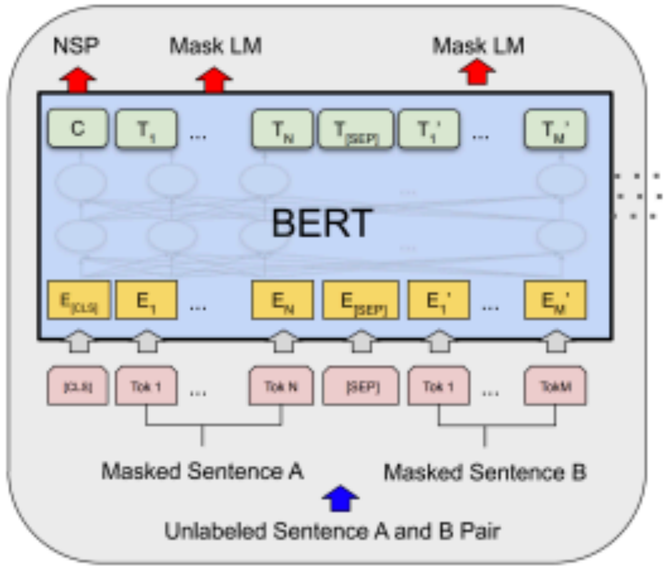
- Input: [CLS] Text chunk 1 [SEP] Text chunk 2
- 50% of the time, take the true next chunk of text, 50% of the time take a random other chunk. Predict whether the next chunk is the “true” next
- BERT objective: masked LM + next sentence prediction

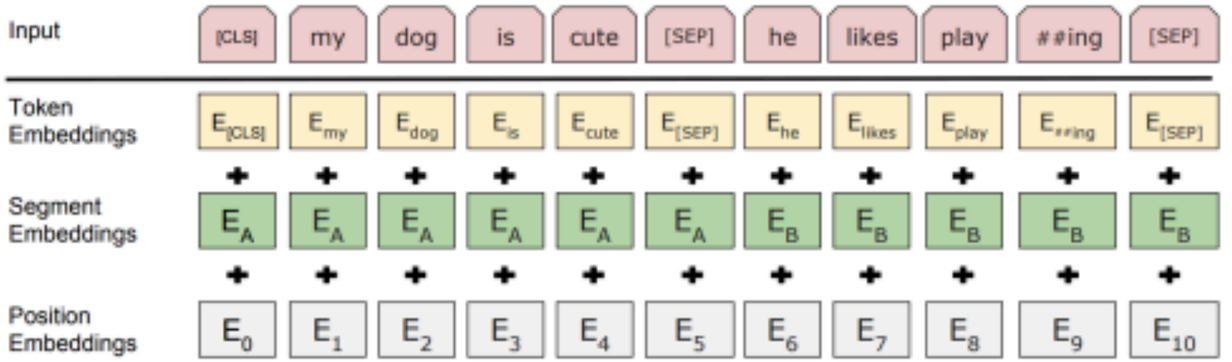


Devlin et al. (2019)

BERT Architecture

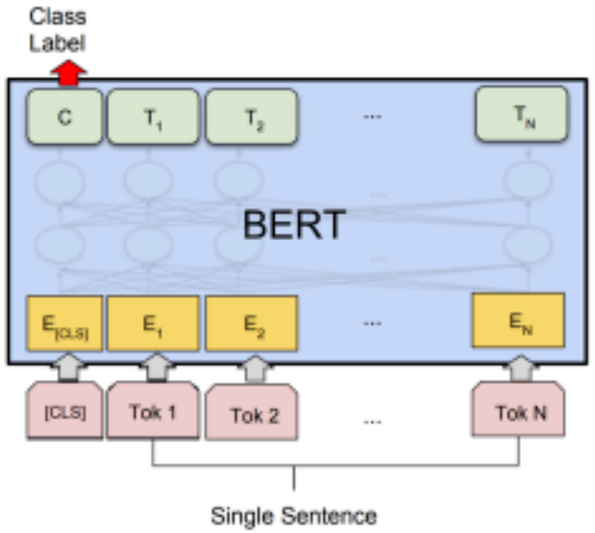
- BERT Base: 12 layers, 768-dim per wordpiece token, 12 heads. Total params = 110M
- BERT Large: 24 layers, 1024-dim per wordpiece token, 16 heads. Total params = 340M
- Positional embeddings and segment embeddings, 30k word pieces
- This is the model that gets **pre-trained** on a large corpus



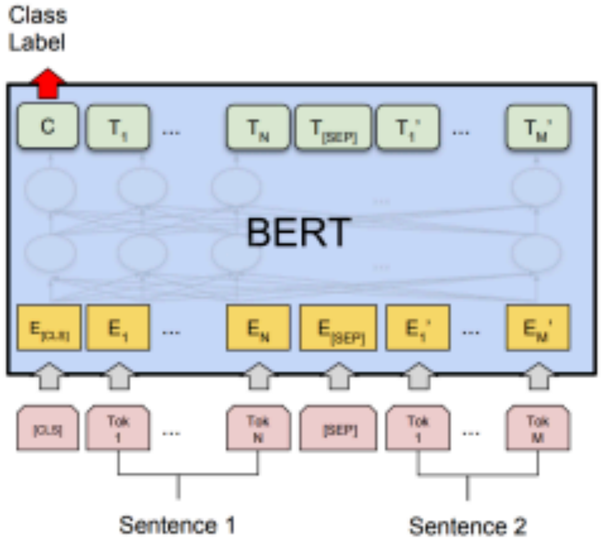


Devlin et al. (2019)

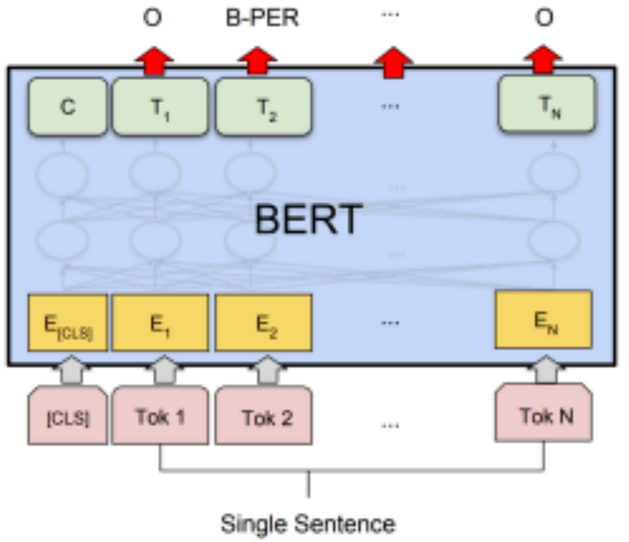
What can BERT do?



(b) Single Sentence Classification Tasks: SST-2, CoLA



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

- Artificial [CLS] token is used as the vector to do classification from
- Sentence pair tasks (entailment): feed both sentences into BERT
- BERT can also do tagging by predicting tags at each word piece

Devlin et al. (2019)

Natural Language Inference

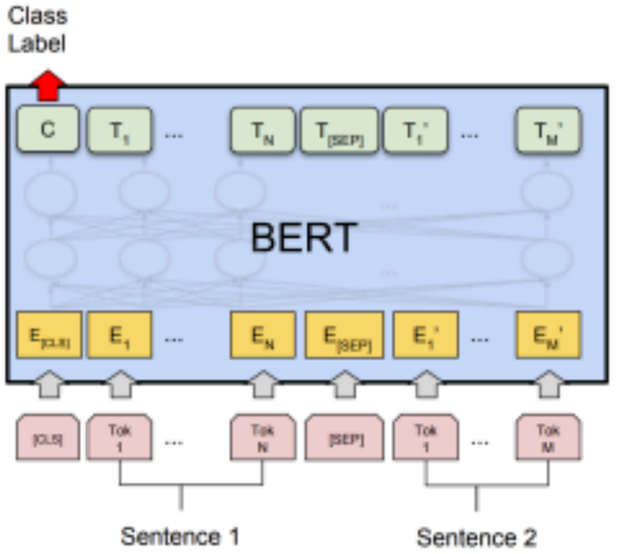
Premise		Hypothesis
A boy plays in the snow	*entails*	A boy is outside
A man inspects the uniform of a figure	*contradicts*	The man is sleeping
An older and younger man smiling	*neutral*	Two men are smiling and laughing at cats playing

- Long history of this task: “Recognizing Textual Entailment” challenge in 2006 (Dagan, Glickman, Magnini)
- Early datasets: small (hundreds of pairs), very ambitious (lots of world knowledge, temporal reasoning, etc.)

What can BERT do?

Entails (first sentence implies second is true)

[CLS] A boy plays in the snow [SEP] A boy is outside



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

- How does BERT model this sentence pair stuff?
- Transformers can capture interactions between the two sentences, even though the NSP objective doesn't really cause this to happen

SQuAD

Q: What was Marie Curie the first female recipient of?

Passage: One of the most famous people born in Warsaw was Marie Skłodowska-Curie, who achieved international recognition for her research on radioactivity and was the first female recipient of the Nobel Prize. Famous musicians include Władysław Szpilman and Frédéric Chopin. Though Chopin was born in the village of Żelazowa Wola, about 60 km (37 mi) from Warsaw, he moved to the city with his family when he was seven months old. Casimir Pulaski, a Polish general and hero of the American Revolutionary War, was born here in 1745.

Answer = Nobel Prize

- Assume we know a passage that contains the answer. More recent work has shown how to retrieve these effectively (will discuss when we get to QA)

SQuAD

Q: What was Marie Curie the first female recipient of?

Passage: One of the most famous people born in Warsaw was Marie Skłodowska-Curie, who achieved international recognition for her research on radioactivity and was the first female recipient of the **Nobel Prize**. …

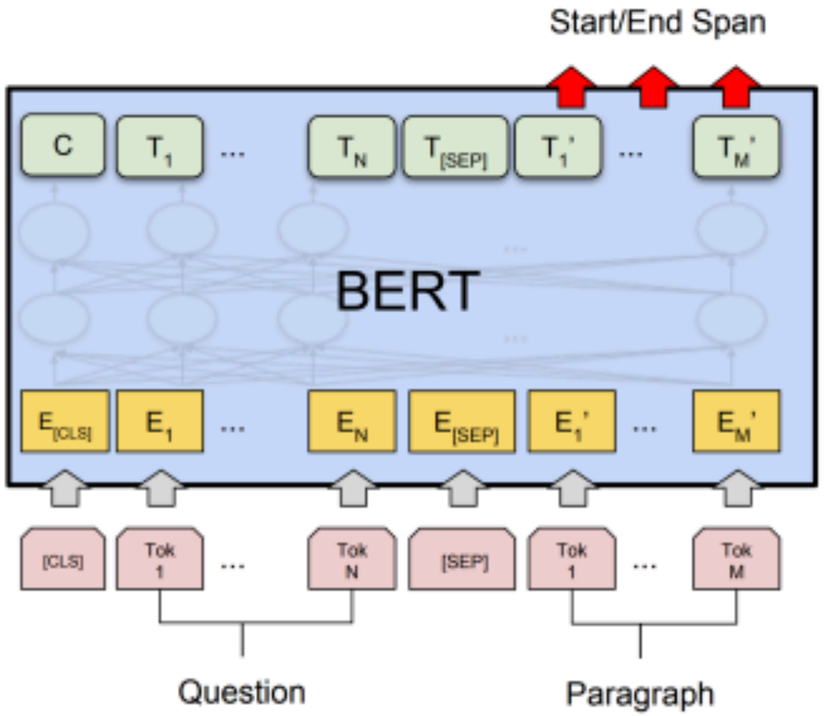
- Predict answer as a pair of (start, end) indices given question q and passage p; compute a score for each word and softmax those

P(start | q, p) =

0.01	0.01	0.01	0.85	0.01
↑	↑	↑	↑	↑
recipient	of	the	**Nobel**	**Prize** .

P(end | q, p) = same computation but different params

QA with BERT



What was Marie Curie the first female recipient of ? [SEP] One of the most famous people born in Warsaw was Marie …

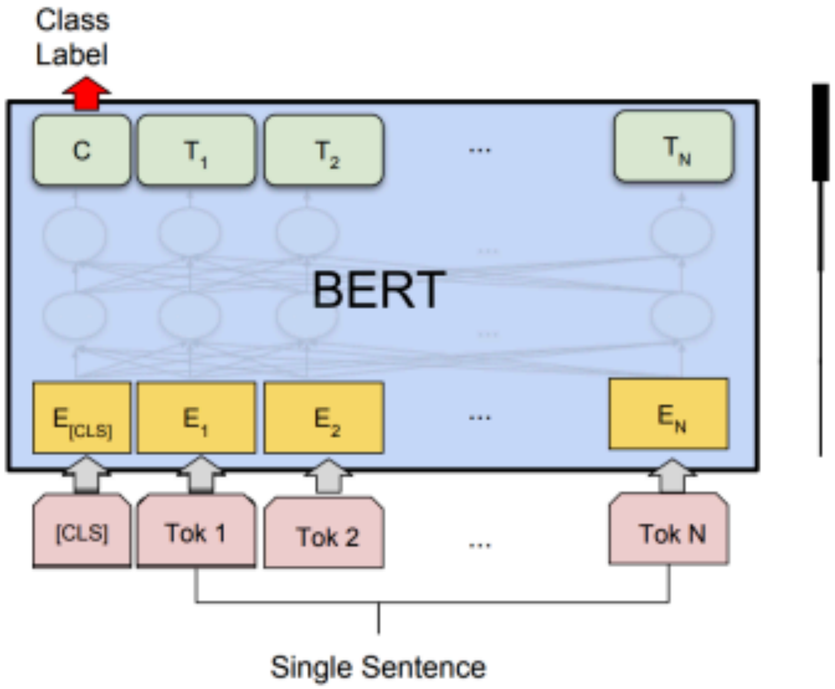
Devlin et al. (2019)

What can BERT NOT do?

- BERT **cannot** generate text (at least not in an obvious way) ‣
  - Can fill in MASK tokens, but can’t generate left-to-right (well, you could put MASK at the end repeatedly, but this is slow)
- Masked language models are intended to be used primarily for “analysis” tasks

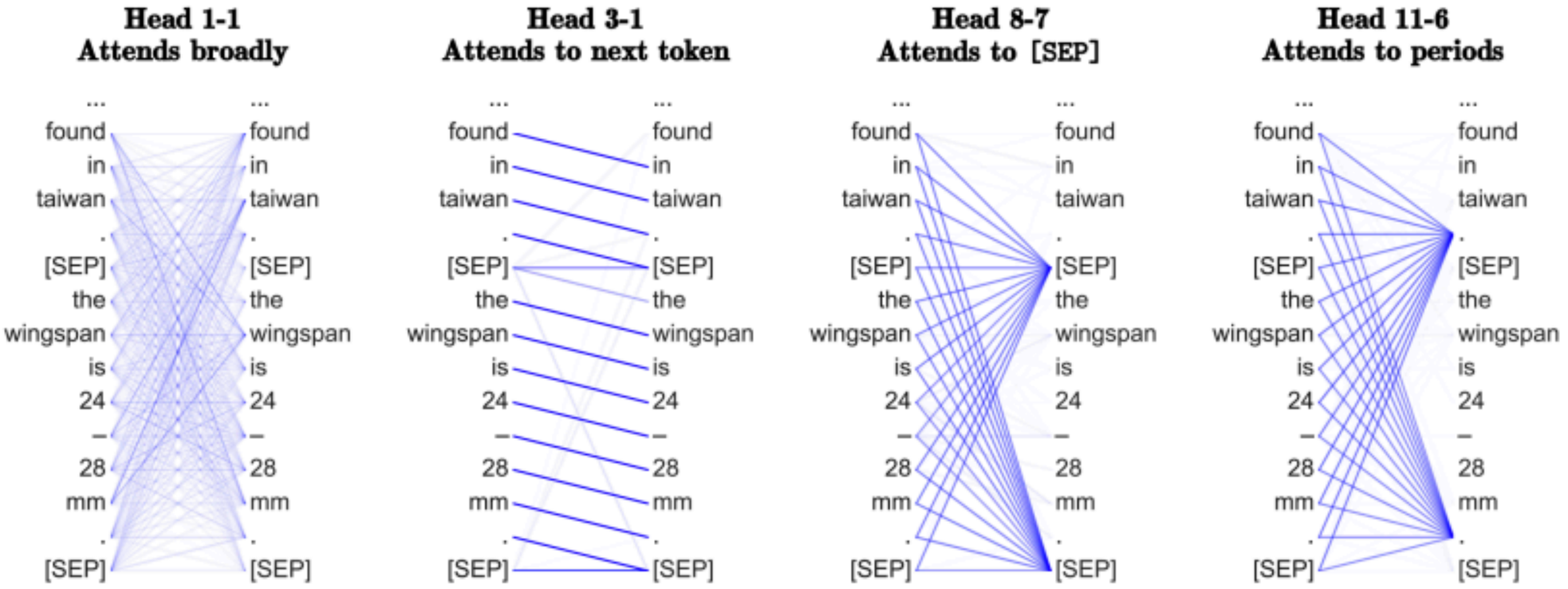
Fine-tuning BERT

- Fine-tune for 1-3 epochs, batch size 2-32, learning rate 2e-5 - 5e-5



(b) Single Sentence Classification Tasks: SST-2, CoLA

- Large changes to weights up here (particularly in last layer to route the right information to [CLS])
- Smaller changes to weights lower down in the transformer
- Small LR and short fine-tuning schedule mean weights don't change much
- Often requires tricky learning rate schedules ("triangular" learning rates with warmup periods)



- Heads on transformers learn interesting and diverse things: content heads (attend based on content), positional heads (based on position), etc.

Clark et al. (2019)

Multiple-Choice Question Answering

Q: What *gas* is primary reason for the "greenhouse effect" on *Earth*?
A. Oxygen B. Carbon dioxide* C. Nitrogen D. Helium E. Argon

(1) Recognize commonalities.
They are all *gases* on *Earth*.

(2) Elinimate commonalities and extract justifications from the question for each choice.
1. The tokens "greenhouse effect" are associated with the choice "Oxygen" and "Carbon dioxide".
2. On the basis of 1, the tokens "primary reason" serve as a justification for the choice "Carbon dioxide".

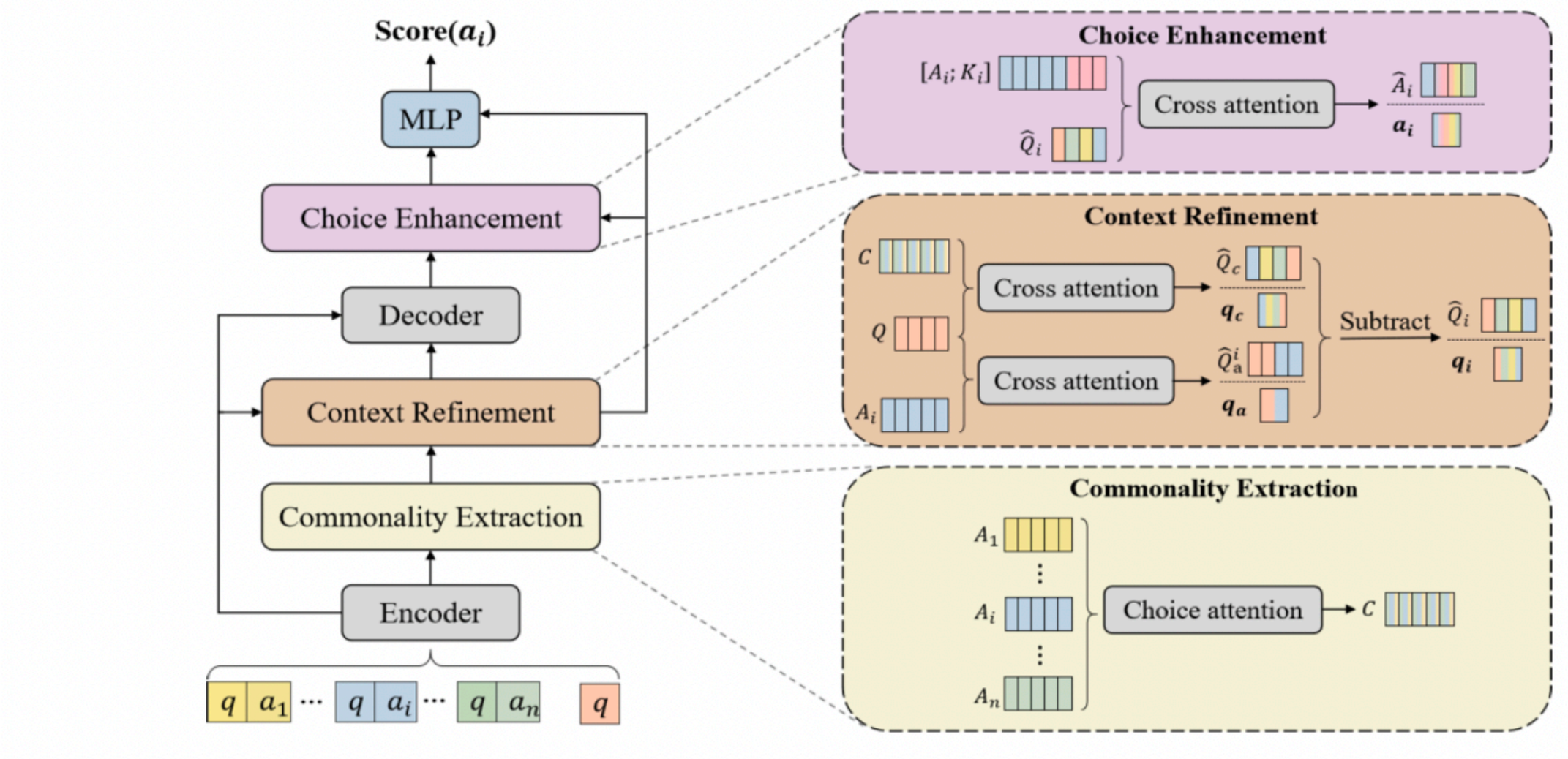
(3) Generate the information from the decoder for each choice.
For choice "Oxygen", the information is "a solution".
For choice "Carbon dioxide", the information is "a cause".
.....

(4) Make selection.
The predicted choice is B.

- An example of MCQA with its reasoning process to derive the correct choice B.
- Red tokens in the question are common to all choices
- Blue tokens differentiate voices

Deng et al 2024 ECAI

Deng et al 2024 ECAI



Deng et al 2024 ECAI

Model	CSQA		OBQA		ARC-E	
	Dev	Test	Dev	Test	Dev	Test
T5-Base						
T5-vanilla	56.59(±0.31)	60.42(±0.54)	59.93(±1.16)	56.60(±1.70)	49.62(±1.17)	50.75(±1.04)
GenMC_{T5}	61.05(±0.40)	**62.87(±0.22)**	**63.00(±0.33)**	61.27(±1.60)	60.43(±0.42)	56.27(±0.61)
Our Model	**61.56(±0.34)**	62.41(±0.24)	62.40(±0.33)	**61.87(±0.62)**	**61.49(±0.68)**	**57.17(±0.60)**
U-T5-Base						
$\text{UnifiedQA}_{\text{T5}}$	42.14(±0.00)	45.05(±0.00)	59.00(±0.00)	55.40(±0.00)	51.32(±0.00)	49.09(±0.00)
$\text{UnifiedQA}_{\text{T5-FT}}$	56.76(±0.80)	59.60(±0.74)	63.00(±0.33)	58.67(±0.53)	55.85(±0.42)	56.19(±0.30)
Our Model	**61.00(±0.64)**	**63.64(±0.31)**	**64.07(±0.34)**	**62.0(±0.99)**	**61.96(±0.54)**	**58.52(±0.31)**
T5-Large						
T5-vanilla	67.26(±0.30)	70.54(±0.78)	65.20(±0.65)	63.13(±1.09)	61.67(±1.09)	59.07(±0.39)
GenMC_{T5}	69.75(±0.04)	72.42(±1.35)	69.10(±0.82)	67.40(±0.75)	70.49(±0.30)	66.55(±0.69)
Our Model	**71.04(±0.30)**	**73.05(±0.27)**	**71.20(±0.43)**	**67.70(±0.47)**	**71.61(±0.63)**	**67.20(±0.17)**
U-T5-Large						
$\text{UnifiedQA}_{\text{T5}}$	56.00(±0.00)	60.85(±0.00)	67.40(±0.00)	67.80(±0.00)	65.61(±0.00)	62.66(±0.00)
$\text{UnifiedQA}_{\text{T5-FT}}$	68.85(±0.04)	71.88(±0.54)	71.33(±0.68)	68.60(±0.16)	71.45(±0.27)	67.56(±0.14)
Our Model	**71.45(±0.68)**	**75.10(±0.70)**	**72.73(±1.20)**	**70.07(±1.36)**	**73.72(±0.50)**	**69.96(±0.35)**

Subword Tokenization

Tokenization Today

- **All pre-trained** models use some kind of subword tokenization with a tuned vocabulary; usually between 50k and 250k pieces (larger number of pieces for multilingual models)

- As a result, classical word embeddings like GloVe **are not used**. All subword representations are randomly initialised and learned in the Transformer models

Handling Rare Words

- Words are a difficult unit to work with. Why?
  - When you have 100,000+ words, the final matrix multiply and softmax start to dominate the computation, many params, still some words you haven’t seen, doesn’t take advantage of morphology, …
- Character-level models were explored extensively in 2016-2018 but simply don’t work well — becomes very expensive to represent sequences

Subword Tokenisation

- Subword tokenization: wide range of schemes that use tokens that are **between characters and words** in terms of granularity
- These “word pieces” may be full words or parts of words

  _the _**eco tax** _port i co _in _Po nt - de - Bu is …

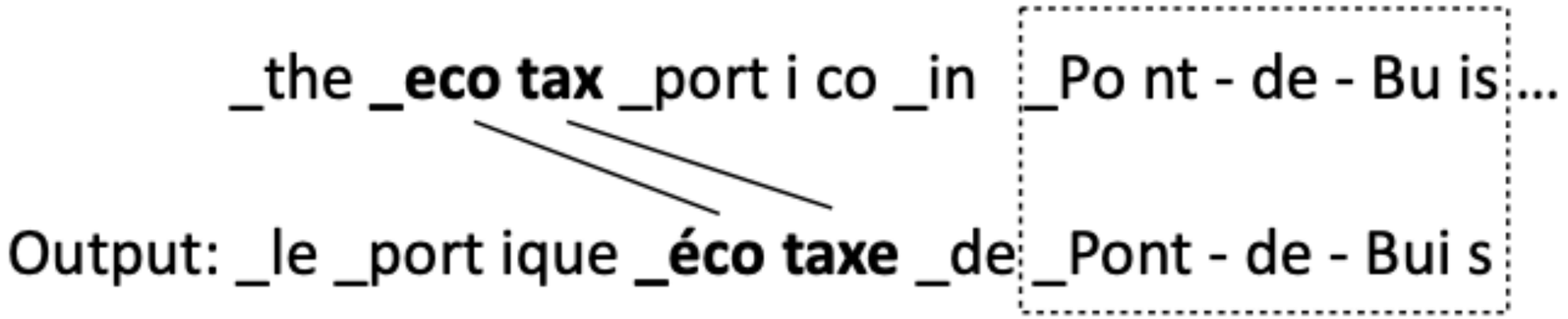
- “_” indicates the word piece starting a word (can think of it as the space character).

Sennrich et al. (2016

Subword Tokenization

- Subword tokenization: wide range of schemes that use tokens that are **between characters and words** in terms of granularity

- These "word pieces" may be full words or parts of words



- Can achieve transliteration with this, subword structure makes some translations easier to achieve

Sennrich et al. (2016)

Byte Pair Encoding (BPE)

- BPE begins with a base vocabulary of individual characters (or bytes) found in the training corpus, plus a special end-of-word symbol </w>
- Every word in the corpus is represented as a sequence of these characters.
  - E.g., the word "low" becomes into 4 bytes: l o w </w>

BPE Algorithm (The Merge Mechanism)

- Iteratively perform the following steps:
    1. Count all adjacent symbol pairs across the entire corpus
    2. Find the most frequent pair (e.g. l o)
    3. Merge that pair into a single new symbol (e.g. lo)
    4. Add the new symbol to the vocabulary
    5. Repeat until the vocabulary reaches a predefined size
- Each merge pass reduces the total number of tokens in the corpus.

Example

- Given corpus: l, o, w, </w>, l, o, w, e, r </w>, n, e, w, e, r, </w>
- BPE Merge:
- er (most frequent pair is added as new symbol)
- er<w/> (most frequent pair is added as new symbol)
- lo (most frequent pair is added as new symbol)
- low (most frequent pair is added as new symbol)

Properties of BPE

- **Frequent words** end up as single tokens; **rare words** are broken into subword pieces
- Out-of-vocabulary words can still be represented by falling back to smaller subword units or characters — the corpus never truly runs out of tokens
- The vocabulary size is a hyperparameter that is set in advance (e.g. 32,000 tokens for many LLMs)
- The learned merge rules are applied deterministically at inference time to new text
- BPE naturally captures morphological structure — prefixes, suffixes, and stems emerge as frequent subwords (e.g. un, ing, er) — giving models meaningful subword units without any linguistic knowledge baked in.

BPE

- Doing 8k merges => vocabulary of around 8000 word pieces, including many whole words
- Many popular LLMs use BPE.
- While BPE is popular, other tokenisation methods are also used, e.g., WordPiece is used by BERT.